

Graph Traversals (BFS & DFS) -- Data Structures

Definition: Graph Traversal is the process of visiting all vertices reachable from a starting vertex in a graph. The two fundamental search algorithms are Breadth-First Search (BFS) and Depth-First Search (DFS).

1. Detailed Technical Explanation

Sample Graph:



1. Breadth-First Search (BFS)

- * Strategy: Explores neighbor vertices level-by-level using a FIFO Queue.

* Alg

2. Depth-First Search (DFS)

- * Strategy: Explores as deep as possible along each branch before backtracking, using Recursion or a Stack.

* Alg

2. Complete Executable C Program for BFS & DFS

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX 10

int adj[MAX][MAX];
bool visited[MAX];
int V = 6;

// 1. Breadth-First Search (BFS)
void BFS(int start) {
    int queue[MAX], front = 0, rear = 0;
    bool visitedBFS[MAX] = false;

    visitedBFS[start] = true;
    queue[rear++] = start;

    printf("BFS Traversal: ");
```

```

while (front < rear) {
    int u = queue[front++];
    printf("%d ", u);

    for (int v = 0; v < V; v++) {
        if (adj[u][v] == 1 && !visitedBFS[v]) {
            visitedBFS[v] = true;
            queue[rear++] = v;
        }
    }
}
printf("\n");
}

// 2. Depth-First Search (DFS)
void DFS(int u) {
    visited[u] = true;
    printf("%d ", u);

    for (int v = 0; v < V; v++) {
        if (adj[u][v] == 1 && !visited[v]) {
            DFS(v);
        }
    }
}

int main() {
    // Construct edges for sample graph
    adj[0][1] = adj[1][0] = 1;
    adj[0][2] = adj[2][0] = 1;
    adj[1][3] = adj[3][1] = 1;
    adj[1][4] = adj[4][1] = 1;
    adj[2][5] = adj[5][2] = 1;

    BFS(0);

    for (int i = 0; i < V; i++) visited[i] = false;
    printf("DFS Traversal: ");
    DFS(0);
    printf("\n");

    return 0;
}

```

3. Comparison of BFS and DFS

Feature	Breadth-First Search (BFS)	Depth-First Search (DFS)
Data Structure	FIFO Queue	Call Stack / Recursion
Search Pattern	Level-by-level exploration	Branch-by-branch deep dive
Time Complexity	$O(V + E)$ (using Adj List)	$O(V + E)$ (using Adj List)
Space Complexity	$O(V)$ (queue size)	$O(V)$ (recursion stack height)
Shortest Path	Guarantees shortest path in unweighted	Does not guarantee shortest path

4. Quick Recall Flow

BFS -> Queue FIFO (Level-by-Level) -> Shortest Path | DFS -> Stack / Recursion (Deep Dive + Backtrack) -> Cycle

Detection